

Batch Normalization: Normalizing Activations

1. Motivation and Benefits

Batch Normalization (Batch Norm), developed by Sergey Ioffe and Christian Szegedy, is an algorithm designed to make training deep neural networks faster and more stable.

- **Key Benefits:**
 - Makes **hyperparameter search** easier and more robust.
 - Allows for a much wider range of functional hyperparameters.
 - Enables the training of very deep networks.
 - **Concept:**
 - Just as normalizing input features (X) speeds up learning in logistic regression by making the cost function contours rounder, Batch Norm extends this concept to hidden layers.
 - **Goal:** Normalize the mean and variance of the inputs to a hidden layer (specifically $Z^{[l]}$) to make the training of parameters $W^{[l+1]}$ and $b^{[l+1]}$ more efficient.
 - **Standard Practice:** While there is debate on whether to normalize Z (pre-activation) or A (post-activation), normalizing Z is much more common and recommended.
-

2. The Batch Norm Algorithm

For a given hidden layer l and a mini-batch of size m , let the intermediate values be $z^{(1)}, \dots, z^{(m)}$. (Note: Layer index $[l]$ is omitted below for brevity).

Step-by-Step Implementation:

1. Compute the Mean:

$$\mu = \frac{1}{m} \sum_{i=1}^m z^{(i)}$$

2. Compute the Variance:

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (z^{(i)} - \mu)^2$$

3. Normalize:

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- ϵ (**epsilon**): Added for numerical stability to prevent division by zero.
 - This step forces the values to have mean 0 and unit variance.
-

3. Learnable Parameters: Gamma and Beta

Strictly forcing hidden units to have mean 0 and variance 1 limits the network's expressiveness (e.g., it might constrain inputs to the linear regime of a sigmoid function). To resolve this, Batch Norm introduces two learnable parameters: γ (**gamma**) and β (**beta**).

- **Scaling and Shifting:**

The final value $\tilde{z}$ is computed as:

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

- **Properties of γ and β :**

- These are **learnable parameters** of the model, updated via gradient descent (similar to weights W and bias b).
- They allow the network to control the mean and variance of the hidden units.
- **Identity Function:** If $\gamma = \sqrt{\sigma^2 + \epsilon}$ and $\beta = \mu$, the operation reverts $\tilde{z}^{(i)}$ back to the original $z^{(i)}$. This ensures the network can undo the normalization if necessary.

4. Integration into the Network

- In the forward propagation steps, replace the original $z^{(i)}$ values with the normalized and scaled values $\tilde{z}^{(i)}$.
 - The subsequent activation function is applied to $\tilde{z}$: $a^{[l]} = g^{[l]}(\tilde{z}^{[l]})$.
-

Batch Normalization: Fitting into Deep Neural Networks

1. Integration Flow in Deep Networks

Batch Normalization (BN) is applied to the values Z (pre-activation) before they are passed to the activation function $g(Z)$.

- **Standard Flow (without BN):**

$$X \xrightarrow{W^{[1]}, b^{[1]}} Z^{[1]} \xrightarrow{g^{[1]}} A^{[1]} \xrightarrow{W^{[2]}, b^{[2]}} Z^{[2]} \dots$$

- **Batch Norm Flow:**

$$X \xrightarrow{W^{[1],b^{[1]}}} Z^{[1]} \xrightarrow{\text{BN}(\beta^{[1]},\gamma^{[1]})} \tilde{Z}^{[1]} \xrightarrow{g^{[1]}} A^{[1]} \xrightarrow{W^{[2],b^{[2]}}} Z^{[2]} \xrightarrow{\text{BN}(\beta^{[2]},\gamma^{[2]})} \tilde{Z}^{[2]} \dots$$

- **Computation:** The network computes $Z^{[l]}$, normalizes it using the current batch's mean/variance, scales/shifts it using $\gamma^{[l]}$ and $\beta^{[l]}$ to get $\tilde{Z}^{[l]}$, and finally applies the activation function to get $A^{[l]}$.

2. Learnable Parameters

Implementing Batch Norm introduces new parameters to the network for each layer l :

- $\gamma^{[l]}$ (**Gamma**): Scale parameter.
- $\beta^{[l]}$ (**Beta**): Shift parameter.

Note on Notation:

- The parameter $\beta^{[l]}$ in Batch Norm is **unrelated** to the hyperparameter β used in Momentum, RMSprop, or Adam optimization (which controls exponentially weighted averages). They share the same symbol but serve completely different purposes.

3. Redundancy of the Bias Term (b)

When Batch Norm is applied, the bias parameter $b^{[l]}$ **becomes redundant**.

- **Reasoning:**

1. Standard computation: $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$.
2. Batch Norm step: The algorithm computes the mean μ of $Z^{[l]}$.
3. Normalization: The step $(Z^{[l]} - \mu)$ subtracts the mean. Since $b^{[l]}$ is a constant added to every example in the mini-batch, it is included in the mean μ .
4. **Cancellation:** Subtracting the mean automatically subtracts $b^{[l]}$, effectively canceling it out.

- **Implementation:**

- You can set $b^{[l]} = 0$ or eliminate it entirely from the parameter set.
- The shifting role previously held by $b^{[l]}$ is replaced by the learnable parameter $\beta^{[l]}$.
- **New Equation:** $Z^{[l]} = W^{[l]}A^{[l-1]}$ (followed by BN steps).

4. Dimensions and Shapes

If layer l has $n^{[l]}$ hidden units:

- $Z^{[l]}$: Shape $(n^{[l]}, m)$ (where m is mini-batch size).
 - $\beta^{[l]}$: Shape $(n^{[l]}, 1)$.
 - $\gamma^{[l]}$: Shape $(n^{[l]}, 1)$.
-

5. Implementing with Mini-Batch Gradient Descent

Batch Norm is typically applied in the context of Mini-Batch Gradient Descent.

Algorithm Loop:

For $t = 1$ to Number of Mini-Batches:

1. Forward Propagation:

- Compute $Z^{[l]}$ on the current mini-batch $X^{\{t\}}$.
- **Apply Batch Norm:** Use the mean and variance of *only* the current mini-batch to compute $\tilde{Z}^{[l]}$.
- Apply activation: $A^{[l]} = g^{[l]}(\tilde{Z}^{[l]})$.

2. Backpropagation:

- Compute gradients $dW^{[l]}, d\beta^{[l]}, d\gamma^{[l]}$ (no $db^{[l]}$ needed).

3. Parameter Update:

- Update $W^{[l]}, \beta^{[l]}, \gamma^{[l]}$ using Gradient Descent, Adam, RMSprop, or Momentum.
- Example update: $\beta^{[l]} := \beta^{[l]} - \alpha d\beta^{[l]}$.

Note: Deep learning frameworks (e.g., TensorFlow, PyTorch) typically handle the complexities of BN implementation (mean/variance calculation) with a single function call.

Why Batch Normalization Works

1. Mitigating "Covariate Shift"

A primary reason for the effectiveness of Batch Normalization is its ability to handle **Covariate Shift** within the hidden layers of a deep neural network.

- **Definition of Covariate Shift:** This occurs when the distribution of input values

(X) changes (e.g., training a model on images of only black cats, but testing it on colored cats). Even if the ground truth mapping ($X \rightarrow Y$) remains the same, the learning algorithm must adapt to the new distribution.

- **Internal Covariate Shift:** In deep networks, the parameters of earlier layers ($W^{[1]}, b^{[1]}$, etc.) change constantly during training.
 - This causes the distribution of activations (the "inputs" to deeper layers) to shift constantly.
 - From the perspective of a deeper layer (e.g., layer 3), its inputs are unstable, forcing it to constantly re-adapt to these changing statistics.
- **The Solution:** Batch Norm limits the amount the distribution of hidden unit values ($Z^{[l]}$) can shift.
 - It constrains $Z^{[l]}$ to have a consistent mean and variance (governed by $\beta^{[l]}$ and $\gamma^{[l]}$).
 - This provides a stable distribution for later layers to learn from, allowing them to learn more independently of the earlier layers.
 - **Result:** It weakens the coupling between layers, speeding up the overall learning process.

2. Similar to Input Normalization

Just as normalizing input features (X) to have mean 0 and variance 1 speeds up learning in Logistic Regression (by making error surface contours rounder), Batch Norm applies this same principle to the hidden units. It ensures that features deep in the network share a similar scale, making optimization easier.

3. Regularization as a Side Effect

Batch Normalization introduces a slight **regularization effect**, similar to Dropout, though this is not its primary intent.

- **Mechanism of Noise:**
 - BN computes the mean (μ) and variance (σ^2) on a **mini-batch** (e.g., 64 or 128 examples) rather than the entire dataset.
 - Since the mini-batch is a small sample, these estimates contain statistical noise.
 - When scaling $Z^{[l]}$ by this noisy σ (multiplicative noise) and subtracting the noisy μ (additive noise), the process injects randomness into the activations.
- **Effect:** This forces downstream hidden units not to rely too heavily on any single upstream unit, preventing overfitting.
- **Comparison to Dropout:**

- Dropout adds noise by multiplying activations by 0 or 1.
 - Batch Norm adds noise via estimation errors in mean and deviation.
 - **Mini-batch Size:** The regularization effect diminishes with larger mini-batch sizes (e.g., 512), as the mean/variance estimates become less noisy.
 - **Recommendation:** Do not use BN *primarily* for regularization; use it for normalization. You can use BN and Dropout together if stronger regularization is needed.
-

Batch Normalization at Test Time

1. The Problem: Training vs. Testing

There is a fundamental difference in how data is processed during training versus test (inference) time when using Batch Normalization.

- **During Training:**
 - Data is processed in **mini-batches** (e.g., 64, 128 examples).
 - The mean (μ) and variance (σ^2) are calculated explicitly from the current mini-batch to normalize the data.
 - **Equations used during training:**
 - $\mu = \frac{1}{m} \sum z^{(i)}$
 - $\sigma^2 = \frac{1}{m} \sum (z^{(i)} - \mu)^2$
 - $z_{norm} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$
 - $\tilde{z} = \gamma z_{norm} + \beta$
- **At Test Time:**
 - You often process examples **one at a time** (or in very small batches).
 - Calculating the mean and variance of a single example makes no sense (the mean would be the value itself, and the variance would be zero).
 - Therefore, you cannot rely on the batch statistics method used during training.

2. The Solution: Estimating Global Statistics

To apply the neural network at test time, you need fixed, independent estimates for μ and σ^2 that represent the training data distribution.

- **Exponentially Weighted Average (Running Average):**

- This is the standard method used to estimate the global mean and variance.
- **Procedure:**
 1. During training, while processing each mini-batch $X^{\{t\}}$, you calculate that batch's $\mu^{\{t\}}$ and $(\sigma^2)^{\{t\}}$.
 2. You maintain a separate **exponentially weighted average** (running average) of these values across all mini-batches.
 3. These running averages track the "global" mean and variance of the hidden layers.
- **Alternative (Less Common):**
 - theoretically, you could run the entire training set through the finalized network to compute the exact global mean and variance, but the running average method is more efficient and standard practice.

3. Implementation at Test Time

When deploying the model (making predictions), you use the **running averages** calculated during training as the constants for normalization.

Steps:

1. **Retrieve Statistics:** Get the final exponentially weighted averages for μ and σ^2 (let's call them μ_{avg} and σ_{avg}^2).
2. **Normalize:** Apply the standard batch norm equation using these fixed values:

$$z_{norm} = \frac{z - \mu_{avg}}{\sqrt{\sigma_{avg}^2 + \epsilon}}$$
3. **Scale and Shift:** Use the learned parameters γ and β (which are static after training):

$$\tilde{z} = \gamma z_{norm} + \beta$$

4. Practical Note

- **Robustness:** The exact method of estimating the mean/variance (running average vs. global calculation) usually matters little; any reasonable estimate of the training data statistics works well.
 - **Deep Learning Frameworks:** If you are using a framework like TensorFlow or PyTorch, this process is automated.
 - The framework automatically tracks the running mean and variance during the training phase.
 - When you switch the model to "eval" or "test" mode, the framework automatically freezes these statistics and uses them for prediction.
-